

20

Transformers

The architecture behind every modern AI model

Built entirely from attention — the engine of GPT and BERT

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	A neural network built entirely from attention
Born in	2017, paper 'Attention Is All You Need'
Key trait	Processes all words in parallel — very fast
Powers	GPT, BERT, Claude, and every large language model
Why it matters	The most important architecture in AI today

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is a Transformer?	The big idea in one minute
2	Why It Beat RNNs	Parallel and never forgets
3	Positional Encoding	Telling the model word order
4	Inside a Transformer Block	The repeating building unit
5	Stacking Blocks	How depth builds understanding
6	Three Families	Encoder, decoder, both (BERT/GPT/T5)
7	Tokens and Embeddings	How text becomes numbers
8	How an LLM Generates Text	One token at a time
9	Using a Transformer in Code	Hugging Face made easy
10	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

3. Positional Encoding

Processing all words at once creates one small problem: if you look at every word simultaneously, you lose track of their ORDER. But order matters — 'dog bites man' and 'man bites dog' are very different!

The fix is **positional encoding** (■■■■■■■■ ■■■■■■■■). The model adds a little 'position tag' to each word, telling it where the word sits in the sentence. Now the model knows both what each word is AND where it is.

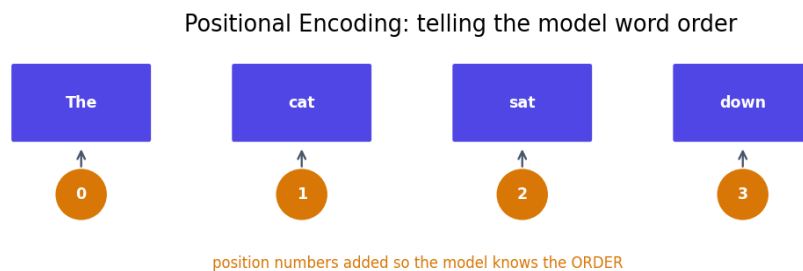
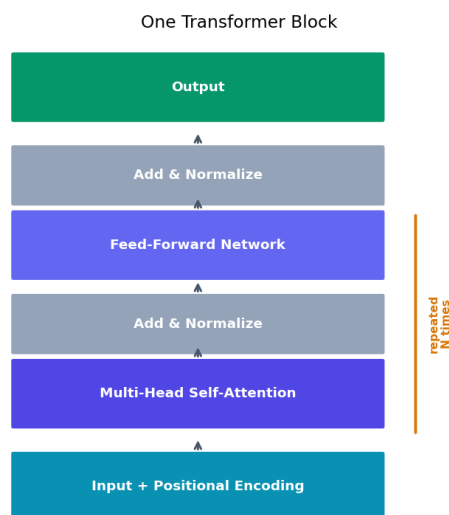


Fig 2 — Each word gets a position number added, so the model knows the order even when looking at all words at once.

■ Simple way to think about it: attention tells the model WHAT to focus on; positional encoding tells it WHERE each word is. Together they give full understanding of the sentence.

4. Inside a Transformer Block

A Transformer is made of identical building units called **blocks** stacked on top of each other. Each block has the same parts. Once you know one block, you know the whole thing.



Part	What it does
Self-Attention	Each word looks at all others for context
Add & Normalize	Keeps the numbers stable and steady
Feed-Forward	A small neural net processes each word
Add & Normalize	Stabilises again before the next block

Fig 3 — One Transformer block. The whole model stacks many of these.

The two stars are **multi-head self-attention** (gather context from all words) and the **feed-forward network** (a small dense network that thinks about each word). The 'Add & Normalize' steps are helpers that keep training smooth. That's the entire block.

5. Stacking Blocks

One block is good; many blocks are powerful. A Transformer stacks the same block over and over — 6, 12, 96, or more times. Each block refines the understanding a little more, just like the layers of a CNN built up from edges to objects.

Early blocks capture simple patterns (grammar, nearby words). Deeper blocks capture meaning, relationships, and even reasoning. The number of blocks is one of the things that makes a model 'bigger' and more capable.

Model	Roughly how many blocks
Original Transformer (2017)	6
BERT-base	12
GPT-3	96
Modern large models	Dozens to over a hundred

■ *More blocks (and more width) means more 'parameters' — the weights the model learns. This is why models are described by size, like '7 billion parameters'. More parameters can learn more, but need more data and compute.*

6. Three Families of Transformers

Transformers come in three flavours, depending on which parts they use. Knowing these helps you recognise any model you meet.

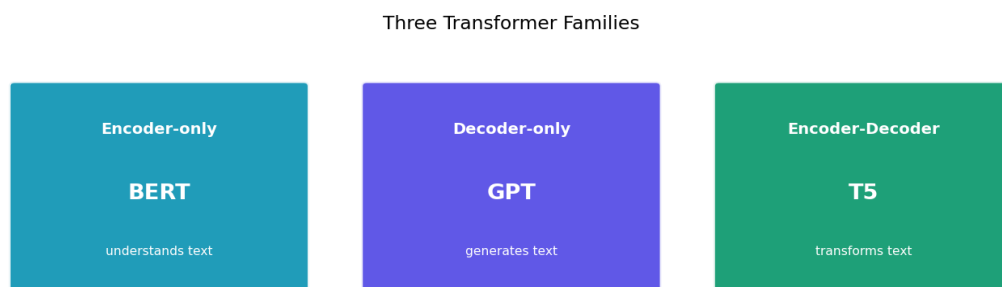


Fig 4 — The three families: encoder-only (BERT), decoder-only (GPT), and encoder-decoder (T5).

Family	Example	Best at
Encoder-only	BERT	Understanding text (search, classify, sentiment)
Decoder-only	GPT, Claude	Generating text (chat, writing, code)
Encoder-Decoder	T5, BART	Transforming text (translation, summarising)

Decoder-only models like GPT are behind most chatbots today — they're built to generate the next word, over and over. **Encoder-only** models like BERT are great when you need to understand a whole piece of text at once. You used encoder-style models in your RAG and sentiment work.

7. Tokens and Embeddings

Transformers don't read whole words exactly. They break text into **tokens** (■■■■■) — small pieces that might be a word, part of a word, or a punctuation mark. 'unbelievable' might become 'un', 'believ', 'able'. This lets the model handle any word, even ones it hasn't seen.

Each token is then turned into an **embedding** (■■■■■■■■■) — a list of numbers that captures its meaning. Words with similar meanings get similar embeddings. This is how text becomes the numbers a Transformer can work with.

```
# Roughly how text becomes model input:
text  = 'I love AI'
tokens = ['I', 'love', 'AI']           # 1. split into tokens
ids    = [40, 1842, 16796]             # 2. each token -> an ID
# 3. each ID -> an embedding vector the model understands
# 4. add positional encoding, then into the blocks
```

■ You hear 'tokens' constantly with LLMs — pricing, context limits, and speed are all measured in tokens. A token is roughly 3/4 of a word on average. Now you know exactly what they are.

8. How an LLM Generates Text

A chatbot like GPT or Claude is a decoder-only Transformer. Here's the surprisingly simple idea behind how it writes: it predicts the next token, one at a time, over and over.

- **Step 1:** Read everything so far (your prompt).
- **Step 2:** Predict the single most likely next token.
- **Step 3:** Add that token to the text.
- **Step 4:** Repeat — now predict the next token, including what it just wrote.
- **Stop:** when it predicts a special 'end' token or hits a length limit.

That's it. A model that writes essays, code, and poems is, at heart, a very good next-token predictor running in a loop. Its power comes from being trained on huge amounts of text, so its 'next token' guesses are remarkably smart.

9. Using a Transformer in Code

You almost never build a Transformer from scratch. You use a pre-trained one from **Hugging Face** (next topics cover this). It's just a few lines:

```
# pip install transformers
from transformers import pipeline

# Sentiment analysis with a pre-trained Transformer
classifier = pipeline('sentiment-analysis')
result = classifier('I love learning about AI!')
print(result)  # [{'label': 'POSITIVE', 'score': 0.99}]

# Text generation
generator = pipeline('text-generation', model='gpt2')
print(generator('The future of AI is', max_length=30))

# Translation
translator = pipeline('translation_en_to_fr')
print(translator('Hello, how are you?'))
```

■ This is the real-world workflow: grab a powerful pre-trained Transformer and use it in a few lines. You did this in your projects. The next topic, Hugging Face, is all about this toolkit.

10. Common Mistakes

Misunderstanding	The reality
'It reads word by word'	It processes all tokens in parallel
'It truly understands'	It's a very good pattern predictor
'Bigger is always better'	Bigger needs more data, compute, and cost
'It remembers past chats'	Only what fits in its context window
'You must build one'	Use pre-trained models from Hugging Face
'Token = word'	A token is often part of a word (~3/4 word)

Quick Reference Cheatsheet

Concept	Meaning
Transformer	a network built from stacked attention blocks
Parallel	processes all words at once (fast)
Positional encoding	adds word-order info

Block	self-attention + feed-forward + normalize
Stacking	many blocks build deeper understanding
Encoder-only (BERT)	understands text
Decoder-only (GPT)	generates text
Encoder-decoder (T5)	transforms text (translate, summarise)
Token	a small piece of text (~3/4 of a word)
Embedding	numbers capturing a token's meaning
Text generation	predict next token, repeat
In code	<code>transformers.pipeline(...)</code>

The one idea to remember: a Transformer stacks many attention blocks to process all words in parallel, using positional encoding to keep order. It never forgets, trains fast, and powers every modern AI model. To generate text, it simply predicts the next token over and over.

Next Topic: Computer Vision — stepping back to the big picture of how machines see and understand images, tying together CNNs and the newer vision Transformers.